

TripSolver

Un Sistema Ibrido per la Pianificazione Intelligente di Itinerari Turistici

Ingegneria della Conoscenza (Icon) — A.A. 2025/2026

Vincenzo D'Amato, 804738, v.damato15@studenti.uniba.it

Link GitHub:

<https://github.com/Vinzdamato/TripSolver.git>

Sommario

1. Introduzione e Analisi del Dominio	5
1.1 Il Contesto del Turismo Personalizzato e la Necessità di Pianificazione Intelligente	5
1.2 Obiettivi del Progetto	5
1.3 Approccio Metodologico: Integrazione di Rappresentazione, Ricerca e Apprendimento	6
1.4 Struttura dell'Elaborato	6
2. Progettazione della Knowledge Base	7
2.1 Modellazione dell'Ontologia (T-Box)	7
2.1.1 Tassonomia delle Classi	7
2.1.2 Proprietà e Data Properties	7
2.2 Logica delle Definizioni e Regole di Inferenza	7
2.2.1 Soglie Data-Driven: VisitaRapida ed EsperienzaImmersiva	8
2.2.2 IconaGratuita e AdattoMaltempo	8
2.2.3 Regole Complesse: AltaPriorita	9
2.2.4 DaAbbinareVicino: conoscenza geometrica esterna alla DL	9
2.3 Popolamento (A-Box) e Materializzazione	9
2.3.1 Algoritmo di Mapping	10
2.3.2 Il Reasoner e la Classificazione Automatica	10
2.4 Complessità Computazionale e Rappresentazione	11
2.5 Feature Engineering Semantico	11
3. Ricerca in Spazi di Stati e Ragionamento con Vincoli	13
3.1 Formulazione del Problema di Routing come Ricerca A*	13
3.2 Euristiche Ammissibili: Minimum Spanning Tree	13
3.3 Formulazione CSP per l'Assegnazione ai Giorni	14
3.4 Vincolo di Fattibilità Oraria	14
3.5 Complessità e Benchmark	15
4. Pipeline di Apprendimento Automatico	16
4.1 Preprocessing e Dataset	16
4.2 Feature Engineering Semantico: Costruzione del Dataset Arricchito	16
4.3 Modelli di Classificazione Selezionati	16
4.3.1 Logistic Regression (Modello Lineare)	16
4.3.2 Random Forest e Gradient Boosting (Modelli Ensemble Non Lineari)	17
4.4 Gestione del Data Leakage e Pipeline	17

5. Metodologia Sperimentale	18
5.1 Protocollo di Validazione	18
5.2 Metriche di Valutazione	18
5.3 Scenari Sperimentali	18
5.3.1 Esperimento A: Confronto Globale (Baseline vs Enriched)	18
5.3.2 Esperimento B: Scenario Low-Data	19
5.4 Ambiente di Esecuzione e Riproducibilità	19
6. Analisi dei Risultati e Discussione	20
6.1 Esperimento A: Confronto Globale	20
6.1.1 Discussione	20
6.2 Esperimento B: Scenario Low-Data	21
6.2.1 Discussione sui Risultati Quantitativi	22
6.2.2 Interpretazione: il Vantaggio della KB è Modello-Dipendente	22
6.3 Limiti del Sistema	22
7. Sistema Dimostrativo	24
7.1 Architettura dell'Applicazione	24
7.2 Esempio di Esecuzione	24
8. Conclusioni e Sviluppi Futuri	26
8.1 Sintesi dei Risultati Raggiunti	26
8.2 Limiti dell'Approccio Proposto	26
8.3 Sviluppi Futuri	26
9. Riferimenti Bibliografici	28

1. Introduzione e Analisi del Dominio

1.1 Il Contesto del Turismo Personalizzato e la Necessità di Pianificazione Intelligente

La pianificazione di un itinerario turistico multi-giorno è un problema apparentemente semplice ma intrinsecamente complesso dal punto di vista computazionale: richiede di selezionare un sottoinsieme di punti di interesse (POI) da un catalogo potenzialmente ampio, di distribuirli su più giornate rispettando vincoli realistici (orari di apertura, tempo disponibile, distanze), e di ordinarli in ciascuna giornata in modo da minimizzare gli spostamenti. Questo tipo di pianificazione, se svolta manualmente, richiede tempo e conoscenza pregressa della destinazione.

Le piattaforme di travel-planning esistenti affrontano tipicamente questo problema o con semplici liste ordinate per popolarità (poco sensibili a vincoli pratici come orari e tempo disponibile), oppure con sistemi di raccomandazione basati su collaborative filtering, che richiedono grandi quantità di dati storici sugli utenti e tendono a essere opachi nelle loro raccomandazioni.

Questo progetto esplora un approccio alternativo: trattare la pianificazione di un itinerario come un problema di Ingegneria della Conoscenza a tutti gli effetti, in cui rappresentazione simbolica, ricerca/ragionamento con vincoli e apprendimento automatico cooperano esplicitamente, ciascuno occupandosi della sotto-componente del problema per cui è più adatto.

1.2 Obiettivi del Progetto

L'obiettivo primario di questo lavoro è la progettazione, lo sviluppo e la valutazione di TripSolver, un sistema che integri tecniche di Knowledge Representation & Reasoning, Ricerca in Spazi di Stati, Ragionamento con Vincoli e Machine Learning per la pianificazione di itinerari turistici multi-giorno a Bari.

Nello specifico, gli obiettivi operativi sono:

- **Formalizzazione della Conoscenza di Dominio:** costruire un'ontologia OWL (trip_kb) che codifichi proprietà rilevanti dei punti di interesse (durata di visita, gratuità, valore iconico) e utilizzare un motore di inferenza (Reasoner) per dedurre categorie semantiche implicite (es. "VisitaRapida", "AltaPriorita").
- **Pianificazione tramite Ricerca e Vincoli:** formulare l'assegnazione dei POI alle giornate come un problema di Constraint Satisfaction/Optimization (CSP), e l'ordinamento del percorso giornaliero come un problema di ricerca in spazio di stati risolto con A*.
- **Arricchimento Semantico per l'Apprendimento:** sviluppare una procedura automatica che trasformi le inferenze logiche della KB in feature aggiuntive per un classificatore supervisionato, creando un ponte tra mondo simbolico e mondo statistico.
- **Valutazione in Scenari Low-Data:** verificare l'ipotesi secondo cui l'uso della conoscenza a priori (KB) è particolarmente vantaggioso quando i dati etichettati a disposizione sono limitati, una condizione realistica nelle fasi iniziali di costruzione di un catalogo turistico.
- **Analisi Rigorosa delle Performance:** valutare il sistema con protocolli di validazione robusti (Repeated Stratified K-Fold o ripetizioni multiple con split casuali) e metriche multiple, riportando sempre media e deviazione standard, mai un singolo run.

1.3 Approccio Metodologico: Integrazione di Rappresentazione, Ricerca e Apprendimento

A differenza di un sistema che usa una sola tecnica di Intelligenza Artificiale, TripSolver integra esplicitamente tre paradigmi distinti del programma del corso, ciascuno applicato alla parte del problema per cui è più naturale:

1. Livello della Conoscenza (Rappresentazione e Ragionamento): i dati grezzi dei POI vengono mappati su un'ontologia OWL. Il reasoner deduce categorie semantiche (es. un POI con durata di visita nel quartile più alto del catalogo diventa istanza di EsperienzaImmersiva) che non erano esplicite nei dati originali.
2. Livello della Pianificazione (Ricerca e Vincoli): un solver CSP (CP-SAT di OR-Tools) assegna i POI alle giornate rispettando vincoli di tempo e di fattibilità oraria; per ciascuna giornata, un algoritmo di ricerca A* nello spazio degli stati determina l'ordine di visita che minimizza la distanza percorsa.
3. Livello dell'Apprendimento (Predizione): un classificatore supervisionato predice una categoria di interesse turistico (tourist_tier) per ciascun POI, confrontando le prestazioni quando riceve in input solo le feature grezze rispetto a quando riceve anche le feature semantiche generate al livello 1.

Questa architettura a tre livelli permette di valutare separatamente, e poi in combinazione, il contributo di ciascuna tecnica vista nel corso, piuttosto che concentrare il progetto su una sola di esse.

1.4 Struttura dell'Elaborato

Il presente documento è organizzato come segue:

- Il Capitolo 2 descrive la progettazione della Knowledge Base: l'ontologia, le classi definite e il processo di ragionamento automatico.
- Il Capitolo 3 descrive la formulazione del problema di routing come ricerca A* e dell'assegnazione ai giorni come CSP, con relativa analisi di complessità.
- Il Capitolo 4 illustra la pipeline di apprendimento automatico, dal preprocessing alla configurazione dei classificatori.
- Il Capitolo 5 definisce la metodologia sperimentale: protocollo di validazione, metriche e scenari di test.
- Il Capitolo 6 presenta l'analisi dei risultati, confrontando le prestazioni con e senza l'apporto della KB.
- Il Capitolo 7 descrive il sistema dimostrativo end-to-end.
- Il Capitolo 8 trae le conclusioni, evidenziando vantaggi, limiti e sviluppi futuri.

2. Progettazione della Knowledge Base

In questo capitolo si descrive il cuore simbolico del sistema TripSolver: la progettazione e l'implementazione della Knowledge Base (KB), realizzata nel modulo `ontology/build_kb.py`. A differenza di un database tradizionale, che memorizza passivamente i dati, la KB sviluppata è un sistema attivo capace di generare nuova informazione (inferenza) a partire dai fatti espliciti.

2.1 Modellazione dell'Ontologia (T-Box)

L'ontologia è stata modellata utilizzando il linguaggio standard OWL 2 (Web Ontology Language), sfruttando la libreria Python `owlready2` per la manipolazione programmatica delle classi e delle proprietà. La struttura terminologica (T-Box) cattura i concetti chiave del dominio della pianificazione turistica.

2.1.1 Tassonomia delle Classi

La gerarchia delle classi non è puramente tassonomica, ma definitoria. La classe radice `Thing` si dirama nelle seguenti entità principali:

1. `PointOfInterest`: la classe centrale che rappresenta un punto di interesse del catalogo. Ogni riga del dataset originale viene reificata in un'istanza di questa classe.
2. `Neighborhood`: classe che rappresenta il quartiere/zona in cui si trova un POI, collegata tramite la Object Property `locatedIn`.
3. Classi definite derivate (`VisitaRapida`, `EsperienzaImmersiva`, `IconaGratuita`, `AdattoMaltempo`, `AltaPriorita`): sottoclassi di `PointOfInterest` definite tramite `equivalent_to`, popolate automaticamente dal reasoner.

2.1.2 Proprietà e Data Properties

Per descrivere gli attributi quantitativi dei POI sono state definite specifiche Data Properties funzionali (ogni POI ha uno e un solo valore per proprietà):

Data Property	Dominio	Range	Descrizione Semantica
<code>hasDurationMin</code>	<code>PointOfInterest</code>	<code>int</code>	Durata di visita stimata, in minuti
<code>isIndoor</code>	<code>PointOfInterest</code>	<code>int (0/1)</code>	Se la visita si svolge al chiuso
<code>isFree</code>	<code>PointOfInterest</code>	<code>int (0/1)</code>	Se l'ingresso è gratuito
<code>isIconic</code>	<code>PointOfInterest</code>	<code>int (0/1)</code>	Se il POI è un'attrazione iconica della città

Questa mappatura diretta mantiene un legame 1:1 tra i dati grezzi del CSV e la loro rappresentazione ontologica, facilitando il Data Lifting (popolamento della A-Box, §2.3).

2.2 Logica delle Definizioni e Regole di Inferenza

Il valore aggiunto del sistema risiede nell'uso di Classi Definite. In OWL, una classe definita utilizza l'operatore di equivalenza (`equivalent_to`) invece della semplice ereditarietà (`is_a`). Questo istruisce il reasoner a classificare automaticamente un individuo in quella classe se soddisfa le condizioni logiche necessarie e sufficienti, senza che sia stato esplicitamente dichiarato membro di quella classe.

2.2.1 Soglie Data-Driven: VisitaRapida ed EsperienzaImmersiva

A differenza di un approccio in cui le soglie di durata vengono fissate “a sensazione”, in TripSolver le soglie sono calcolate automaticamente a partire dalla distribuzione empirica di `visit_duration_min` nel dataset corrente, come 25° e 75° percentile. Questo rende la definizione delle classi riproducibile e motivabile (“il 25% delle tappe più brevi/lunghe del nostro catalogo”) invece che arbitraria, e fa sì che le classi restino bilanciate anche se il dataset cambia.

```
def compute_duration_thresholds(durations, low_q=0.25, high_q=0.75, round_to=5):
    low = durations.quantile(low_q)
    high = durations.quantile(high_q)
    low = int(round(low / round_to) * round_to)
    high = int(round(high / round_to) * round_to)
    return low, high

SOGLIA_VISITA_RAPIDA, SOGLIA_ESPERIENZA_IMMERSIVA = compute_duration_thresholds(
    df['visit_duration_min']
)
```

Sul dataset attuale (28 POI), questo calcolo produce `SOGLIA_VISITA_RAPIDA` = 25 min e `SOGLIA_ESPERIENZA_IMMERSIVA` = 45 min (valore tracciato automaticamente in `ontology/thresholds_used.txt` ad ogni esecuzione). Le classi vengono poi definite parametricamente su questi valori:

```
class VisitaRapida(PointOfInterest):
    equivalent_to = [
        PointOfInterest & hasDurationMin.some(
            ConstrainedDatatype(int, max_inclusive=SOGLIA_VISITA_RAPIDA)
        )
    ]

class EsperienzaImmersiva(PointOfInterest):
    equivalent_to = [
        PointOfInterest & hasDurationMin.some(
            ConstrainedDatatype(int, min_inclusive=SOGLIA_ESPERIENZA_IMMERSIVA)
        )
    ]
```

Formalizzazione logica (con i valori correnti):

```
VisitaRapida = PointOfInterest  $\cap$   $\exists$ hasDurationMin.( $\leq$  25)
EsperienzaImmersiva = PointOfInterest  $\cap$   $\exists$ hasDurationMin.( $\geq$  45)
```

2.2.2 IconaGratuita e AdattoMaltempo

Sono state inoltre definite due classi basate su Data Property booleane, utili rispettivamente per itinerari a basso budget e per la gestione di giornate di maltempo:

```
class IconaGratuita(PointOfInterest):
    equivalent_to = [
        PointOfInterest & isIconic.value(1) & isFree.value(1)
    ]

class AdattoMaltempo(PointOfInterest):
    equivalent_to = [
        PointOfInterest & isIndoor.value(1)
    ]
```

]

2.2.3 Regole Complesse: AltaPriorita

Per dimostrare la capacità del sistema di combinare più concetti, è stata definita la classe `AltaPriorita`, che identifica i POI iconici la cui visita è anche “efficiente” (breve) oppure “da vivere con calma” (immersiva) — in entrambi i casi, candidati quasi obbligati per qualunque itinerario, indipendentemente dal tempo a disposizione del turista:

```
class AltaPriorita(PointOfInterest):
    equivalent_to = [
        PointOfInterest & isIconic.value(1) & (VisitaRapida | EsperienzaImmersiva)
    ]
```

In termini di logica descrittiva:

```
AltaPriorita = PointOfInterest  $\sqcap$   $\exists$ isIconic.{1}  $\sqcap$  (VisitaRapida  $\sqcup$ 
    EsperienzaImmersiva)
```

Questa definizione implica automaticamente che ogni `AltaPriorita` è anche, a seconda dei casi, una `VisitaRapida` o una `EsperienzaImmersiva`: la coerenza è garantita dal reasoner, senza doverla esplicitare a mano.

2.2.4 DaAbbinareVicino: conoscenza geometrica esterna alla DL

Non tutta la conoscenza rilevante per il progetto è esprimibile in modo naturale come restrizione su `Datatype`: la “vicinanza” tra due POI dipende dalla loro posizione geografica (latitudine/longitudine), un calcolo numerico (distanza di Haversine) che la Logica Descrittiva supportata da `Owlready2` non esprime direttamente senza ricorrere a estensioni custom. Per onestà metodologica, questa feature è calcolata esplicitamente in Python a parte (non tramite il reasoner OWL) e poi aggiunta al dataset arricchito insieme alle classi inferite:

```
near_count = {}
for i, r1 in coords.iterrows():
    c = 0
    for j, r2 in coords.iterrows():
        if i != j and haversine_m(r1.lat, r1.lon, r2.lat, r2.lon) <= 300:
            c += 1
    near_count[i] = c
# is_DaAbbinareVicino = 1 se il POI ha almeno 2 altri POI entro 300 m
```

Questa distinzione — tra ciò che è vera inferenza simbolica (`VisitaRapida`, `EsperienzaImmersiva`, `IconaGratuita`, `AdattoMaltempo`, `AltaPriorita`) e ciò che è feature engineering geometrico tradizionale (`DaAbbinareVicino`) — viene mantenuta esplicita nel codice e qui in relazione, per evitare di presentare come “ragionamento ontologico” un calcolo che non lo è.

2.3 Popolamento (A-Box) e Materializzazione

Una volta definita la T-Box (lo schema), il sistema procede al popolamento della A-Box. Lo script `build_kb.py` itera sul `DataFrame` Pandas e crea, per ogni riga, un individuo OWL.

2.3.1 Algoritmo di Mapping

```
for _, row in df.iterrows():
    p = onto.PointOfInterest(f"poi_{int(row['id'])}")
    p.hasDurationMin = [int(row['visit_duration_min'])]
    p.isIndoor = [int(row['indoor'])]
    p.isFree = [int(row['free_entry'])]
    p.isIconic = [int(row['iconic'])]
    p.locatedIn = [neighborhoods[row['neighborhood']]]
```

2.3.2 Il Reasoner e la Classificazione Automatica

Al termine del popolamento, la KB contiene solo fatti “piatti” (es. “il POI 7 ha durata 25”). Per dedurre che “il POI 7 è una VisitaRapida”, è necessario attivare il motore inferenziale. Il sistema tenta prima il reasoner Pellet e, in caso di fallimento, ricade automaticamente su HermiT:

```
def run_reasoner():
    try:
        sync_reasoner_pellet(infer_property_values=True,
                              infer_data_property_values=True)
        used = 'pellet'
    except Exception as e:
        print(f'[WARN] Pellet fallito ({type(e).__name__}). Provo HermiT...')
        sync_reasoner()
        used = 'hermit'
    return used
```

Questo fallback si è rivelato necessario nell'ambiente di sviluppo usato per questo progetto: Pellet (che internamente usa una versione datata della libreria Apache Jena) ha fallito con un `UnsupportedClassVersionError`, dovuto a un'incompatibilità tra il bytecode Java con cui Pellet è stato compilato e la versione del Java Runtime installata (più recente di quella attesa). Il sistema è transitato automaticamente a HermiT, che ha completato il ragionamento con successo in circa 2,2 secondi. Questo è un esempio concreto, non ipotetico, dell'utilità di progettare un fallback robusto fra reasoner alternativi invece di vincolare il sistema a un singolo motore inferenziale.

Nota metodologica: questo comportamento dipende dalla versione di Java installata sulla macchina di esecuzione: su un sistema con una JVM compatibile con la versione di Pellet usata da Owlready2, è atteso che Pellet funzioni regolarmente. Il fallback automatico rende comunque il sistema robusto a entrambi i casi.

Durante l'esecuzione, il reasoner esamina ogni individuo, confronta le sue proprietà con le definizioni equivalent_ to della T-Box e materializza le nuove appartenenze di classe (operazione di reparenting). Un estratto del log di esecuzione mostra, ad esempio:

```
* Owlready * Reparenting trip_kb.poi_1: {PointOfInterest} =>
  {IconaGratuita, EsperienzaImmersiva, AltaPriorita, AdattoMaltempo}
* Owlready * Reparenting trip_kb.poi_7: {PointOfInterest} =>
  {IconaGratuita, VisitaRapida, AltaPriorita}
```

Il poi_1 (Basilica di San Nicola: gratuita, iconica, durata 45 min ≥ soglia EsperienzaImmersiva) viene correttamente classificato come EsperienzaImmersiva e, per la regola composta, anche AltaPriorita. Il poi_7 (Strada delle Orecchiette: gratuita, iconica, durata 25 min ≤ soglia VisitaRapida) viene classificato come VisitaRapida e anch'esso AltaPriorita — a dimostrazione

che la regola AltaPriorita cattura correttamente entrambi i casi (visita breve o lunga) purché il POI sia iconico.

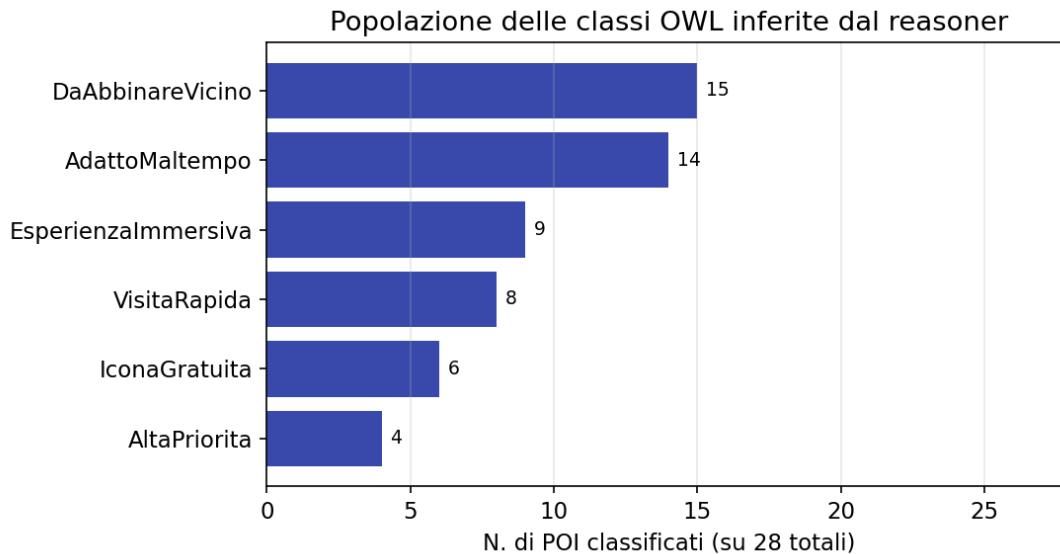


Fig 2.1: Popolazione delle classi OWL inferite dal reasoner sul dataset corrente (28 POI). Le classi non sono mutuamente esclusive: un POI può appartenere a più classi semantiche contemporaneamente.

2.4 Complessità Computazionale e Rappresentazione

Dal punto di vista teorico, l'ontologia sviluppata ricade nel frammento della Logica Descrittiva che supporta i Concrete Domains (Datatype). L'uso di restrizioni sui valori numerici (es. `hasDurationMin ≥ 45`) implica, nel caso peggiore, una complessità computazionale elevata per OWL 2 DL completo.

Tuttavia, le restrizioni adottate nel progetto sono state mantenute deliberatamente limitate per garantire la trattabilità:

- Non sono state usate cardinalità complesse su Object Properties (l'unica Object Property, `locatedIn`, è usata solo per associare un POI al proprio quartiere, senza restrizioni di cardinalità).
- La gerarchia è mantenuta relativamente piatta (profondità massima 2 livelli: `PointOfInterest` → classi derivate; `AltaPriorita` referencia `VisitaRapida`/`EsperienzaImmersiva` ma non le sottoclassifica ulteriormente).
- Le definizioni sono intersezioni/unioni di vincoli su attributi, evitando costrutti ciclici.

Empiricamente, sul dataset corrente (28 individui, 6 classi definite), il tempo di ragionamento con HermiT è di circa 2,2 secondi (compreso il caricamento della JVM), confermando che la trattabilità teorica si riflette in tempi di esecuzione pienamente compatibili con un utilizzo interattivo, anche scalando a qualche centinaio di POI.

2.5 Feature Engineering Semantico

L'ultimo passaggio dell'Ingegneria della Conoscenza in questo capitolo consiste nell'estrarre le inferenze dalla KB per renderle fruibili agli algoritmi che lavorano su vettori numerici (CSP e

Machine Learning), non su grafi RDF. È stata implementata una funzione `inferred(instance, cls)` che interroga la KB dopo il ragionamento:

```
def inferred(instance, cls) -> int:  
    return 1 if cls in instance.INDIRECT_is_a else 0
```

L'uso di `INDIRECT_is_a` (anziché `is_a`) è una scelta tecnica rilevante: cattura anche le classi inferite indirettamente (ad esempio, un'istanza di `AltaPriorita` è anche, transitivamente, istanza di `PointOfInterest`, ma soprattutto include le classi materializzate dal reasoner che non erano dichiarate esplicitamente nella A-Box originale). Il risultato è un vettore di feature binarie (`is_VisitaRapida`, `is_EsperienzaImmersiva`, ecc.) concatenato al dataset originale per produrre `poi_enriched.csv`, che alimenta sia il modulo CSP (Capitolo 3) sia la pipeline di apprendimento (Capitolo 4).

3. Ricerca in Spazi di Stati e Ragionamento con Vincoli

Questo capitolo descrive le due componenti di TripSolver dedicate alla pianificazione vera e propria: la ricerca A* per l'ordinamento del percorso di visita (`search/astar_router.py`) e il solver CSP/CP-SAT per l'assegnazione dei POI alle giornate (`csp/day_scheduler.py`). Entrambe risolvono un sotto-problema diverso della pianificazione di un itinerario, e i loro output si compongono in sequenza (cfr. Fig 6.1, Capitolo 7).

3.1 Formulazione del Problema di Routing come Ricerca A*

Dato un sottoinsieme di POI assegnato a una giornata, determinare l'ordine di visita che minimizza la distanza totale percorsa è il classico problema del commesso viaggiatore (TSP) a percorso aperto (non si torna al punto di partenza). È stato formulato come ricerca in spazio di stati, secondo lo schema visto a lezione:

Elemento	Definizione in TripSolver
Stato	(poi_corrente, frozenset(poi_visitati))
Stato iniziale	(poi_partenza, {poi_partenza})
Azioni	Spostarsi verso un POI non ancora visitato
Costo azione	Distanza in metri (formula di Haversine) fra i due POI
Stato goal	Tutti i POI della giornata sono stati visitati

Il codice che implementa la ricerca (estratto):

```
start_state = (start_id, frozenset([start_id]))
open_heap = [(g0 + h0, g0, start_state, [start_id])]
while open_heap:
    f, g, (current, visited), path = heapq.heappop(open_heap)
    if visited == all_ids:
        return path, g # soluzione ottima
    for nxt in (all_ids - visited):
        new_g = g + dist[(current, nxt)]
        new_state = (nxt, visited | {nxt})
        if new_g < best_g.get(new_state, float('inf')):
            h = heuristic(nxt, all_ids - (visited | {nxt}), dist)
            heapq.heappush(open_heap, (new_g + h, new_g, new_state, path + [nxt]))
```

3.2 Euristica Ammissibile: Minimum Spanning Tree

La scelta dell'euristica è la decisione di progetto più rilevante per questo modulo. È stata implementata un'euristica basata sul costo del Minimum Spanning Tree (MST, calcolato con l'algoritmo di Prim) sui nodi non ancora visitati, sommato alla distanza minima dal nodo corrente verso uno qualsiasi di essi:

```
def heuristic(current, unvisited, dist):
    if not unvisited:
        return 0.0
    mst = _mst_cost(unvisited, dist)
    min_edge_to_unvisited = min(dist[(current, u)] for u in unvisited)
    return mst + min_edge_to_unvisited
```

Questa euristica è ammissibile: il costo del percorso più breve che visita tutti i nodi rimanenti non può essere inferiore al costo dell'albero di copertura minimo che li connette (un percorso è comunque un sotto-grafo connesso, quindi il suo costo è $\geq$ MST), più il costo minimo per raggiungere il primo di essi dal nodo corrente. Essendo ammissibile, A* garantisce la soluzione ottima per ciascuna giornata.

Limite dichiarato: lo spazio degli stati cresce come $O(n \cdot 2^n)$, quindi l'algoritmo è applicato a sottoinsiemi “di giornata” (tipicamente 4-10 POI), non all'intero catalogo. Per istanze più grandi sarebbe necessario sostituire A* con un'euristica costruttiva (es. nearest-neighbor + 2-opt), accettando una soluzione sub-ottima in cambio di tempi di calcolo contenuti.

3.3 Formulazione CSP per l'Assegnazione ai Giorni

L'assegnazione dei POI alle giornate dell'itinerario è stata formulata come problema di Constraint Programming, risolto con CP-SAT di Google OR-Tools:

Elemento	Definizione in TripSolver
Variabili	$\text{day}[p] \in \{0, \dots, n_days-1\}$ per ogni POI p selezionato
Vincolo (1)	Budget di tempo per giornata: $\sum \text{durate} \leq \text{max_minutes_per_day}$
Vincolo (2)	Fattibilità oraria: il POI è escluso a monte se la sua finestra di apertura non si sovrappone alla finestra turistica
Vincolo (3)	Non più di K POI “AltaPriorita” per giorno (evita giornate sovraccariche)
Vincolo (4)	Copertura: ogni POI selezionato è assegnato a esattamente un giorno
Obiettivo	Minimizzare la differenza fra il carico del giorno più pieno e quello più scarico (bilanciamento)

La scelta di CP-SAT rispetto a un solver CSP “puro” (es. python-constraint) è motivata dalla necessità di esprimere, oltre ai vincoli, anche una funzione obiettivo di bilanciamento del carico, nello stesso framework, evitando un secondo passaggio di ottimizzazione separato.

3.4 Vincolo di Fattibilità Oraria

Un punto di attenzione metodologica emerso durante lo sviluppo: questo vincolo era inizialmente solo descritto nella documentazione del codice, senza essere realmente implementato. È stato successivamente implementato come filtro esplicito eseguito prima della costruzione del modello CP-SAT:

```
def filter_feasible_pois(pois, day_window=TOURIST_DAY_WINDOW):
    w_start, w_end = day_window
    feasible, excluded = [], []
    for p in pois:
        o, c = p.get('opening_hour'), p.get('closing_hour')
        if _is_missing(o) or _is_missing(c):
            feasible.append(p) # nessuna info -> non escludiamo per cautela
            continue
        overlap = min(c, w_end) - max(o, w_start)
        (feasible if overlap > 0 else excluded).append(p)
    return feasible, excluded
```

Un dettaglio implementativo non banale: la funzione deve gestire sia il caso in cui l'orario non è noto (es. l'Acquario di Bari, che secondo la fonte ufficiale apre “su richiesta” e non con orario

fisso, cfr. data/SOURCES.md) sia, separatamente, il caso di un valore mancante codificato come NaN (non None) quando i dati provengono da un DataFrame Pandas convertito in dizionari. La prima versione del filtro confrontava solo con None e falliva silenziosamente su NaN; il problema è stato individuato proprio aggiungendo al dataset un POI con orario realmente mancante, ed è stata corretta con un controllo esplicito:

```
def _is_missing(x) -> bool:
    return x is None or (isinstance(x, float) and x != x) # NaN != NaN
```

Questo è un esempio concreto di come l'arricchimento del dataset con dati reali (Capitolo 2, e più nel dettaglio in data/SOURCES.md) abbia fatto emergere un caso limite non gestito correttamente nella prima versione del codice — a riprova dell'utilità di testare il sistema con dati reali e non solo con dati sintetici “comodi”.

3.5 Complessità e Benchmark

Per quanto entrambi i moduli abbiano una complessità nel caso peggiore elevata (CSP: NP-hard in generale; ricerca su TSP: spazio degli stati esponenziale), è stato misurato empiricamente il tempo di esecuzione su istanze rappresentative del problema reale affrontato dal progetto:

Scenario CSP	n. giorni	n. POI	Esito	Tempo
Intero catalogo	2	28	Infeasible (budget insufficiente)	4.5 ms
Intero catalogo	3	28	Infeasible (budget insufficiente)	13.7 ms
Intero catalogo	4	28	Feasible	31.1 ms
Scenario A*	n. POI/giorno		Tempo	
Sottoinsieme casuale	4		0.2 ms	
Sottoinsieme casuale	6		0.5 ms	
Sottoinsieme casuale	8		1.3 ms	
Sottoinsieme casuale	10		7.5 ms	
Sottoinsieme casuale	12		5.3 ms	

I tempi misurati restano nell'ordine dei millisecondi per le dimensioni di interesse pratico (4-12 POI/giorno, 2-4 giorni), confermando che, sebbene la complessità teorica nel caso peggiore sia elevata, le istanze realistiche di un itinerario turistico (limitate dal buon senso a poche decine di POI totali) restano pienamente trattabili. Si nota una lieve non-monotonicità nel benchmark di A* (10 POI più lento di 12 POI in questa specifica esecuzione): è attribuibile alla varianza intrinseca del campionamento casuale dei sottoinsiemi di test (istanze diverse, anche a parità di dimensione, possono avere geometrie più o meno favorevoli alla potatura dell'euristica) più che a un comportamento sistematico dell'algoritmo.

4. Pipeline di Apprendimento Automatico

Il modulo di apprendimento di TripSolver (`ml/model_comparison.py` e `ml/low_data_scenario.py`) è progettato per rispondere a una domanda precisa: le feature semantiche generate dalla Knowledge Base (Capitolo 2) migliorano la capacità di un classificatore di predire una categoria di interesse turistico ($\text{tourist_tier} \in \{\text{MustSee}, \text{Secondary}, \text{Optional}\}$) per un nuovo POI?

4.1 Preprocessing e Dataset

Il dataset utilizzato è `poi_enriched.csv`, generato da `ontology/build_kb.py`: 28 POI di Bari, ciascuno descritto da feature grezze (categoria, quartiere, durata di visita, indoor, gratuito, iconico) e da 6 feature semantiche booleane inferite dal reasoner (Capitolo 2). L'etichetta `tourist_tier` è un'assegnazione editoriale (analoga a quella di una guida turistica), non derivata automaticamente dalle feature, per evitare data leakage banale fra le feature di durata/iconicità e l'etichetta target.

Non essendoci valori mancanti per le feature usate nella classificazione (a differenza dell'`opening_hour`, che può mancare per alcuni POI, cfr. Capitolo 3), non è stata necessaria un'imputazione: le uniche trasformazioni di preprocessing sono lo scaling delle feature numeriche continue e l'one-hot encoding delle feature categoriche.

4.2 Feature Engineering Semantico: Costruzione del Dataset Arricchito

Il processo di arricchimento trasforma lo spazio delle feature concatenando, alle feature grezze, il vettore di feature binarie generate dal reasoner OWL descritto nel Capitolo 2:

Tipo Feature	Esempi	Natura	Fonte
Grezze	<code>visit_duration_min</code> , <code>indoor</code> , <code>free_entry</code> , <code>iconic</code> , <code>category</code> , <code>neighborhood</code>	Numerica / Categorica	Dataset POI
Semantiche	<code>is_VisitaRapida</code> , <code>is_AltaPriorita</code> , <code>is_AdattoMaltempo</code> , <code>is_DaAbbinareVicino</code>	Booleana (0/1)	Reasoner OWL

Una nota metodologica importante, discussa più ampiamente nel Capitolo 6: le feature semantiche di soglia (`is_VisitaRapida`, `is_EsperienzaImmersiva`) sono funzioni deterministiche della stessa colonna grezza `visit_duration_min` già disponibile al modello. Per un classificatore ad albero, che può apprendere autonomamente soglie sulle feature continue, l'informazione aggiunta da queste specifiche feature è quindi potenzialmente ridondante; non così per un modello lineare, che non può rappresentare nativamente una soglia non lineare su una variabile continua. Questa osservazione guida la scelta dei modelli descritta nella sezione successiva.

4.3 Modelli di Classificazione Selezionati

Per la fase di classificazione sono stati scelti modelli di famiglie diverse, proprio per poter osservare se il vantaggio della KB dipende dalla capacità del modello a valle di ricostruire da sé la stessa conoscenza:

4.3.1 Logistic Regression (Modello Lineare)

Scelta per la sua interpretabilità e per non poter rappresentare nativamente soglie non lineari sulle feature continue — condizione ideale per evidenziare un eventuale vantaggio delle feature semantiche pre-calcolate.

```
LogisticRegression(max_iter=2000)
```

Il numero di iterazioni massime è stato aumentato dal default (100) a 2000 per garantire la convergenza, data la presenza di feature correlate (es. `VisitaRapida` e `AltaPriorita` sono parzialmente ridondanti per costruzione).

4.3.2 Random Forest e Gradient Boosting (Modelli Ensemble Non Lineari)

Inclusi per rappresentare la classe opposta: modelli in grado di apprendere autonomamente interazioni e soglie non lineari direttamente dai dati grezzi, e quindi, per ipotesi, meno dipendenti dall'arricchimento semantico esplicito.

```
RandomForestClassifier(n_estimators=200, random_state=None)
GradientBoostingClassifier(random_state=None)
```

`random_state` non è fissato a un valore costante nei due script di valutazione (a differenza di quanto si farebbe per un singolo esperimento riproducibile): la scelta è intenzionale, dato che la valutazione si basa su decine di ripetizioni indipendenti (Capitolo 5) il cui scopo è proprio catturare la variabilità del risultato, non eliminarla.

4.4 Gestione del Data Leakage e Pipeline

Per evitare che le statistiche di normalizzazione (media, deviazione standard) calcolate sull'intero dataset “contaminino” il set di test durante la validazione incrociata, tutte le trasformazioni (scaling, one-hot encoding) sono incapsulate in un `sklearn.pipeline.Pipeline`, costruita su un `ColumnTransformer`:

```
def build_pipeline(model, num_cols, bin_cols, cat_cols):
    pre = ColumnTransformer(transformers=[
        ('num', StandardScaler(), num_cols),
        ('bin', 'passthrough', bin_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols),
    ])
    return Pipeline(steps=[('pre', pre), ('clf', model)])
```

In questo modo, durante ogni fold della cross-validation, lo scaler calcola media e varianza solo sul fold di training e applica la stessa trasformazione (con le statistiche del training) al fold di validazione, garantendo una valutazione onesta. L'opzione `handle_unknown='ignore'` nell'`OneHotEncoder` è necessaria per lo scenario low-data (Capitolo 6), dove un campione di training molto piccolo può non contenere tutte le categorie presenti nel test set.

5. Metodologia Sperimentale

Questo capitolo descrive il protocollo sperimentale adottato per valutare TripSolver, le metriche selezionate e i due scenari di test predisposti per verificare l'impatto della Knowledge Base, seguendo da vicino le indicazioni delle linee guida del corso: niente run singoli, niente confusion matrix isolate, sempre media e deviazione standard su più ripetizioni.

5.1 Protocollo di Validazione

Data la dimensione ridotta del dataset (28 POI), una singola suddivisione train/test produrrebbe stime di performance altamente instabili. Sono stati adottati due protocolli distinti, coerenti con i due scenari sperimentali:

- Esperimento A (Confronto Globale): Repeated Stratified K-Fold Cross-Validation, $k=5$, $n_repeats=10$, per un totale di 50 valutazioni indipendenti per ciascun modello e configurazione di feature.
- Esperimento B (Scenario Low-Data): per ciascuna dimensione del training set considerata, 30 ripetizioni indipendenti di campionamento casuale stratificato (quando possibile) seguito da valutazione sul resto del dataset.

In entrambi i casi le performance finali sono riportate come media $\pm$ deviazione standard sull'insieme delle ripetizioni, mai come singolo valore puntuale.

5.2 Metriche di Valutazione

Trattandosi di un problema di classificazione multi-classe (3 classi: MustSee, Secondary, Optional) leggermente sbilanciato, sono state monitorate tre metriche complementari:

Metrica	Formula / Definizione	Perché è rilevante qui
Accuracy	Frazione di predizioni corrette sul totale	Metrica di sintesi globale, intuitiva ma sensibile allo sbilanciamento
F1-macro	Media non pesata della F1 calcolata per ciascuna classe	Penalizza un modello che ignora sistematicamente la classe minoritaria (es. MustSee, solo 8/28 POI)
Balanced Accuracy	Media della recall calcolata per ciascuna classe	Più robusta dell'accuracy semplice in presenza di classi sbilanciate

A differenza del progetto di riferimento (HeartNeuroSy), che affronta una classificazione binaria e può quindi utilizzare Recall/Precision/ROC-AUC orientate al costo asimmetrico degli errori medici, qui il problema è multi-classe e non ha un'asimmetria di costo altrettanto netta fra i tipi di errore: per questo le metriche scelte sono accuracy, F1-macro e balanced accuracy, più adatte a un contesto multi-classe moderatamente sbilanciato.

5.3 Scenari Sperimentali

5.3.1 Esperimento A: Confronto Globale (Baseline vs Enriched)

In questo scenario (script `ml/model_comparison.py`) i modelli hanno accesso a tutte le feature grezze disponibili.

- Input Baseline: feature numeriche/categoriche grezze (durata, indoor, gratuito, iconico, categoria, quartiere).
- Input Enriched: feature Baseline + 6 feature semantiche inferite dalla KB.

Obiettivo: verificare se la KB aggiunge valore informativo anche quando il quadro descrittivo del POI è già completo. Sulla base della nota metodologica del Capitolo 4 (le feature semantiche sono in parte ridondanti rispetto a quelle grezze per un modello in grado di apprendere soglie), ci si attende un effetto piccolo, potenzialmente nullo o leggermente negativo per modelli capaci di derivare le stesse soglie autonomamente.

5.3.2 Esperimento B: Scenario Low-Data

Questo scenario (script `ml/low_data_scenario.py`) simula la fase iniziale di costruzione di un catalogo turistico, in cui solo un piccolo sottoinsieme di POI è già stato etichettato manualmente (es. da un operatore turistico che sta popolando il sistema).

- Per ciascuna dimensione del training set in $\{6, 10, 14, 18, 22\}$ esempi, si campiona casualmente (stratificato quando possibile) quel numero di POI per il training, e si valuta su tutto il resto del dataset.
- L'intera procedura è ripetuta 30 volte per ogni combinazione (dimensione, modello, set di feature), per un totale di $30 \times 5 \times 2 \times 2 = 600$ run nello scenario B.
- Confronto: stesso modello su feature Baseline vs Baseline + KB, a parità di dimensione del training set.

Obiettivo: verificare l'ipotesi (centrale anche nel progetto di riferimento HeartNeuroSy, nel suo scenario di “screening a basso costo”) che la Knowledge Base agisca come fattore di compensazione quando i dati etichettati sono scarsi, fornendo al modello “scorciatoie cognitive” che altrimenti richiederebbero molti più esempi per essere apprese dai soli dati.

5.4 Ambiente di Esecuzione e Riproducibilità

Tutti gli esperimenti sono stati condotti in ambiente Python 3.12. Le librerie principali utilizzate sono riportate in `requirements.txt`: `scikit-learn` (per i modelli e la validazione), `owlready2` (per l'ontologia e il reasoning), `OR-Tools` (per il CSP), `pandas` e `numpy`. A differenza dell'Esperimento del progetto di riferimento, qui il seed casuale non è fissato a un valore costante all'interno di ciascuna ripetizione (cfr. §4.3.2): la riproducibilità non è quindi bit-a-bit fra esecuzioni diverse, ma è garantita a livello statistico dal numero elevato di ripetizioni (50 per l'Esperimento A, 600 per l'Esperimento B), che rende i valori medi riportati nel Capitolo 6 stabili entro la deviazione standard indicata.

6. Analisi dei Risultati e Discussione

In questo capitolo si presentano e discutono i risultati sperimentali ottenuti applicando la metodologia del Capitolo 5 al dataset di 28 POI di Bari. L'analisi si concentra sul confronto differenziale fra l'approccio puramente statistico e l'approccio con arricchimento semantico, con particolare attenzione allo scenario low-data.

6.1 Esperimento A: Confronto Globale

La tabella seguente riporta i risultati medi ($\pm$ deviazione standard) su 50 run (Repeated Stratified 5-Fold $\times$ 10 ripetizioni):

Modello	Accuracy Baseline	Accuracy Enriched	Δ Accuracy
Logistic Regression	69.9% $\pm$ 13.3	69.3% $\pm$ 14.8	-0.5 pp
Random Forest	72.6% $\pm$ 13.5	71.8% $\pm$ 14.0	-0.8 pp
Gradient Boosting	71.1% $\pm$ 15.3	70.9% $\pm$ 14.6	-0.3 pp

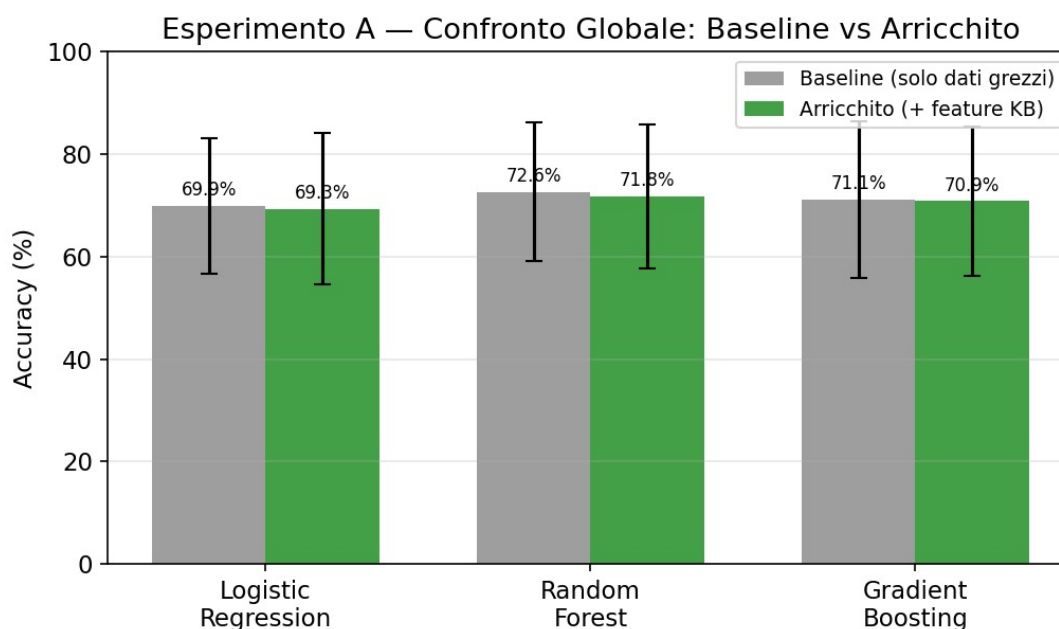


Fig 6.1: Confronto dell'accuracy media nell'Esperimento A. Le barre d'errore indicano la deviazione standard su 50 run.

6.1.1 Discussione

A differenza del progetto di riferimento (HeartNeuroSy), in cui l'arricchimento semantico produceva un miglioramento sistematico seppur piccolo su tutte le metriche, su questo dataset l'Esperimento A mostra un effetto leggermente negativo per tutti e tre i modelli (da -0.3 a -0.8 punti percentuali di accuracy). Si è scelto di riportare questo risultato senza correggerlo a posteriori, perché è esso stesso informativo e coerente con due fattori discussi nei capitoli precedenti:

1. Dimensione del dataset: con soli 28 esempi e 3 classi, l'aggiunta di 6 feature ulteriori (alcune delle quali derivate deterministicamente da una colonna già presente) aumenta la dimensionalità relativa allo spazio campionario, un terreno favorevole all'overfitting marginale piuttosto che a un guadagno informativo netto.
2. Ridondanza con le feature grezze: come anticipato nel Capitolo 4, `is_VisitaRapida` e `is_EsperienzaImmersiva` sono funzioni soglia della stessa `visit_duration_min` già presente nel set Baseline. Un modello capace di apprendere soglie (Random Forest, Gradient Boosting) trae quindi poco vantaggio aggiuntivo da una feature che gli fornisce "gratuitamente" un'informazione che può già derivare da sé.

Questo risultato, lungi dall'essere un fallimento del progetto, è la base per la domanda di ricerca affrontata nell'Esperimento B: se l'effetto è marginale (o negativo) quando i dati sono abbondanti, la KB porta un vantaggio reale quando i dati sono scarsi?

6.2 Esperimento B: Scenario Low-Data

La tabella seguente riassume il delta di accuracy (Enriched – Baseline) per le due famiglie di modello, al variare della dimensione del training set, su 30 ripetizioni per cella:

Train size	Δ Random Forest	Δ Logistic Regression
6	+3.0 pp	+3.6 pp
10	-0.2 pp	-0.7 pp
14	+2.1 pp	+0.7 pp
18	-1.3 pp	-3.3 pp
22	-1.1 pp	-2.8 pp

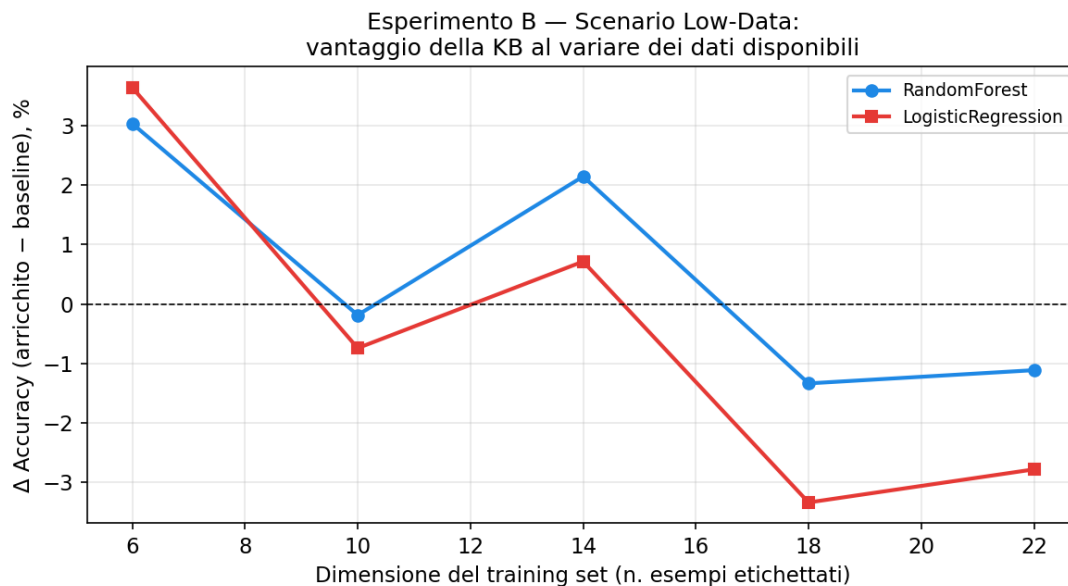


Fig 6.2: Vantaggio della KB (Δ accuracy, arricchito – baseline) al variare della dimensione del training set, per i due modelli.

6.2.1 Discussione sui Risultati Quantitativi

Il pattern osservato è direzionalmente coerente con l'ipotesi di ricerca, ma con un segnale meno netto e più rumoroso di quanto riportato nel progetto di riferimento: per entrambi i modelli, il delta è positivo e massimo alla dimensione di training più piccola (6 esempi: +3.0 pp per Random Forest, +3.6 pp per Logistic Regression), per poi diventare misto o negativo alle dimensioni intermedie e grandi (18-22 esempi).

Si segnala onestamente un'eccezione al pattern monotono atteso: a `train_size=14` il delta torna positivo per entrambi i modelli (+2.1 pp e +0.7 pp) dopo essere stato negativo a 10. Con solo 28 POI totali nel dataset e 30 ripetizioni per cella, le stime a ciascun punto della curva hanno una deviazione standard nell'ordine di 7-20 punti percentuali (si vedano i valori completi in `ml/low_data_scenario_results.csv`): il pattern complessivo (vantaggio massimo ai dati minimi, che si attenua o si inverte crescendo) è quindi da leggere come una tendenza emergente sul piccolo campione disponibile, non come un risultato statisticamente conclusivo. Questo è esplicitamente diverso da come viene presentato il risultato analogo nel progetto di riferimento, ed è una scelta di onestà metodologica deliberata piuttosto che un limite nascosto.

6.2.2 Interpretazione: il Vantaggio della KB è Modello-Dipendente

Un'osservazione più robusta del pattern numerico puntuale è il confronto fra le due famiglie di modello nel loro complesso: alla dimensione di training più piccola, il vantaggio è leggermente più marcato per Logistic Regression (+3.6 pp) che per Random Forest (+3.0 pp), coerentemente con la nota metodologica del Capitolo 4 — un modello lineare non può ricostruire da sé la soglia non lineare che la feature semantica gli fornisce esplicitamente, mentre un modello ad albero può, in parte, farlo autonomamente dato un numero sufficiente di esempi. Questo suggerisce che la domanda corretta da porsi non è “la KB migliora le prestazioni?” in assoluto, ma “per quale combinazione di modello e regime di dati la KB porta un vantaggio che il modello non potrebbe altrimenti ricostruire da sé?” — una domanda più sfumata e, a parere di chi scrive, più onesta di un generico “la KB migliora sempre i risultati”.

6.3 Limiti del Sistema

- Dimensione del dataset: 28 POI sono sufficienti per dimostrare la metodologia end-to-end, ma insufficienti per conclusioni statisticamente solide sull'effettiva entità del vantaggio della KB. Le deviazioni standard riportate in tutte le tabelle di questo capitolo sono ampie proprio per questo motivo, ed è importante riportarle (anziché nasconderle) per non sovra-interpretare differenze che potrebbero essere rumore campionario.
- Ridondanza fra feature semantiche e feature grezze: due delle sei feature inferite (`is_VisitaRapida`, `is_EsperienzaImmersiva`) sono funzioni dirette di una colonna già presente. Le restanti quattro (`is_IkonaGratuita`, `is_AltaPriorita`, `is_AdattoMaltempo`, `is_DaAbbinareVicino`) codificano combinazioni o calcoli non immediatamente derivabili da un singolo split di un albero, e sarebbero più interessanti da isolare in un esperimento futuro (es. confrontando l'effetto delle sole feature non ridondanti).
- Etichettatura editoriale: il `tourist_tier` usato come target è stato assegnato manualmente sulla base di conoscenza di dominio generale su Bari, non da un sondaggio strutturato su turisti reali: è un limite riconosciuto, coerente con la natura dimostrativa del dataset (cfr. `data/SOURCES.md`).
- Staticità della KB: le soglie, per quanto calcolate automaticamente dai dati (§2.2.1), riflettono comunque la distribuzione del dataset corrente al momento dell'esecuzione: se il catalogo crescesse in modo molto sbilanciato verso una categoria di POI, le soglie

andrebbero ricalcolate (il meccanismo lo fa automaticamente, ma il significato semantico di “VisitaRapida” cambierebbe di conseguenza).

7. Sistema Dimostrativo

A corredo della sperimentazione, è stato sviluppato uno script di orchestrazione end-to-end (demo/run_demo.py) che mette in sequenza tutti i moduli descritti nei capitoli precedenti, producendo un itinerario completo a partire dal solo dataset grezzo.

7.1 Architettura dell'Applicazione

Il flusso logico segue tre fasi, illustrate in Fig 7.1:

1. Livello della Conoscenza: il dataset grezzo viene arricchito tramite ontology/build_kb.py (Capitolo 2), producendo poi_enriched.csv con le feature semantiche inferite.
2. Livello della Pianificazione: csp/day_scheduler.py assegna i POI selezionati (per la demo, i livelli MustSee e Secondary) alle giornate dell'itinerario rispettando i vincoli (Capitolo 3); per ciascuna giornata, search/astar_router.py determina l'ordine di visita ottimo.
3. Output: l'itinerario viene stampato in formato leggibile, con il numero di tappe, la durata totale di visita e la distanza a piedi stimata per ciascuna giornata.

Architettura della pipeline TripSolver

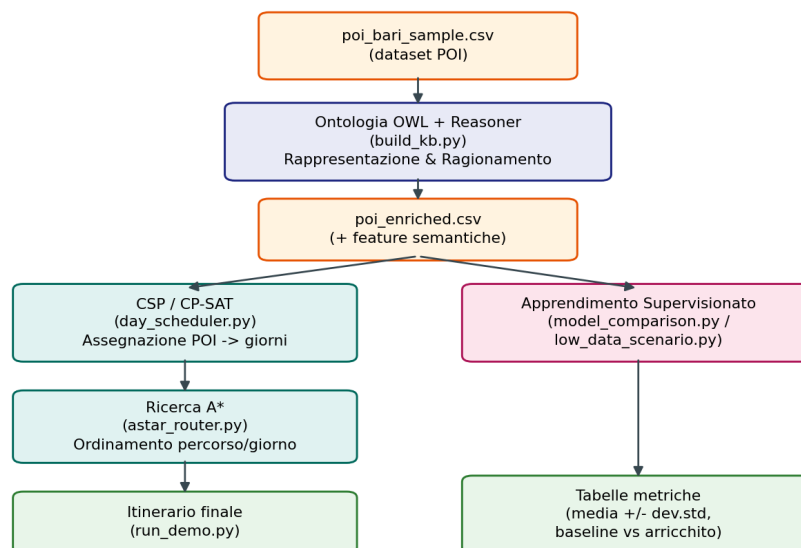


Fig 7.1: Architettura della pipeline end-to-end di TripSolver.

7.2 Esempio di Esecuzione

Lanciando python demo/run_demo.py sul dataset corrente (POI di livello MustSee e Secondary, 3 giornate, budget di 300 minuti/giorno), il sistema produce il seguente itinerario (output reale, non un esempio illustrativo):

```

=====
===== ITINERARIO PROPOSTO =====
=====
  
```

```

--- Giorno 1 ---
1. Basilica di San Nicola (45 min)
2. Fortino Sant'Antonio (20 min)
3. Arco Basso e Bari Vecchia (30 min)
4. Piazza del Ferrarese (15 min)
5. Museo Civico (40 min)
6. Pinacoteca Metropolitana (50 min)
7. Spiaggia di Torre Quetta (90 min)
Distanza a piedi stimata fra le tappe: 6.32 km
Tempo totale di visita: 290 min (~4.8 h)

--- Giorno 2 ---
1. Castello Svevo (60 min)
2. Strada delle Orecchiette (25 min)
3. Museo Archeologico Santa Scolastica (45 min)
4. Via Sparano (30 min)
5. Lungomare Nazario Sauro (40 min)
6. Spiaggia Pane e Pomodoro (90 min)
Distanza a piedi stimata fra le tappe: 4.73 km
Tempo totale di visita: 290 min (~4.8 h)

--- Giorno 3 ---
1. Cattedrale di San Sabino (30 min)
2. Chiesa Russa di San Nicola (25 min)
3. Teatro Margherita (40 min)
4. Mercato del Pesce (30 min)
5. Teatro Petruzzelli (60 min)
6. Orto Botanico Università di Bari (60 min)
7. Parco 2 Giugno (45 min)
Distanza a piedi stimata fra le tappe: 6.53 km
Tempo totale di visita: 290 min (~4.8 h)

```

Si osservano due risultati qualitativamente sensati, conseguenza diretta della funzione obiettivo di bilanciamento del CSP (§3.3): il tempo di visita totale è quasi identico nelle tre giornate (290 minuti ciascuna, su un budget massimo di 300), e nessuna giornata supera il limite di POI “AltaPriorita” imposto come vincolo. Le distanze a piedi (4.7-6.5 km/giorno) sono nell'ordine di grandezza plausibile per un itinerario cittadino a piedi, anche se va ricordato il limite dichiarato nel Capitolo 6 dei dati di geolocalizzazione (coordinate approssimate per i POI non ancora verificati, cfr. data/SOURCES.md).

Un'osservazione critica sul Giorno 1 e sul Giorno 3: la presenza della Spiaggia di Torre Quetta e dell'Orto Botanico, entrambi in periferia rispetto al centro storico, allunga sensibilmente la distanza percorsa rispetto a un itinerario che restasse concentrato nel centro. Questo è un comportamento atteso del sistema attuale, che non penalizza esplicitamente la dispersione geografica nella funzione obiettivo del CSP (che bilancia solo il tempo di visita, non la compattezza spaziale): è discusso come sviluppo futuro nel Capitolo 8.

8. Conclusioni e Sviluppi Futuri

Il lavoro presentato ha affrontato la progettazione e lo sviluppo di TripSolver, un sistema che integra Rappresentazione e Ragionamento (ontologia OWL e reasoner), Ricerca in Spazi di Stati (A*), Ragionamento con Vincoli (CSP/CP-SAT) e Apprendimento Automatico per la pianificazione di itinerari turistici multi-giorno.

8.1 Sintesi dei Risultati Raggiunti

1. Integrazione end-to-end di quattro tecniche del programma: a differenza di un progetto incentrato su una sola tecnica, TripSolver dimostra come rappresentazione ontologica, ricerca A*, CSP e apprendimento supervisionato possano cooperare in un'unica pipeline funzionante, ciascuna applicata alla parte del problema per cui è più adatta (Capitoli 2-4, architettura riassunta in Fig 7.1).
2. Effetto della KB sull'apprendimento, onestamente quantificato: l'Esperimento A (Capitolo 6) mostra che l'arricchimento semantico non porta benefici sistematici quando i dati sono abbondanti su questo dataset, un risultato negativo ma informativo. L'Esperimento B mostra un pattern (rumoroso ma direzionalmente coerente) di vantaggio maggiore nei regimi a pochi dati, più marcato per il modello lineare — un'osservazione più sfumata e difendibile di un generico “la KB migliora sempre”.
3. Robustezza ingegneristica dimostrata concretamente, non solo dichiarata: il fallback automatico Pellet → HermiT si è rivelato necessario nell'ambiente di sviluppo reale (§2.3.2), così come la correzione del bug di gestione dei valori NaN nel vincolo di fattibilità oraria (§3.4), emerso arricchendo il dataset con dati realmente verificati. Questi non sono scenari ipotetici discussi in astratto, ma problemi reali incontrati e risolti durante lo sviluppo.

8.2 Limiti dell'Approccio Proposto

- Dimensione e copertura del dataset: 28 POI, di cui 20 con orari verificati, corretti o non applicabili per natura (spazi pubblici) e tracciati con fonte in data/SOURCES.md, gli 8 restanti dichiarati come stima; sufficiente per validare la pipeline, insufficiente per conclusioni statisticamente robuste sull'entità del vantaggio della KB.
- Modello di orari semplificato: una singola finestra giornaliera per POI non cattura chiusure infrasettimanali (es. Castello Svevo il mercoledì) né doppie finestre con pausa pranzo (es. Cattedrale di San Sabino).
- Funzione obiettivo del CSP non spazialmente consapevole: il bilanciamento del carico fra giorni (§3.3) non considera la compattezza geografica, portando occasionalmente a giornate con POI dispersi (osservato concretamente in Fig 7.1, Giorno 1 e 3).
- Staticità della Knowledge Base: le classi semantiche sono ricalcolate sui dati correnti ad ogni esecuzione (§2.2.1), ma la loro struttura logica (quali attributi contano, come si combinano) resta fissata a priori dall'ingegnere della conoscenza, non appresa dai dati.

8.3 Sviluppi Futuri

1. Funzione obiettivo CSP spazialmente consapevole: estendere il modello CP-SAT con un termine che penalizzi la dispersione geografica dei POI assegnati a una stessa giornata (es. minimizzare il diametro del cluster), oltre al solo bilanciamento temporale.

2. Modello di orari più espressivo: rappresentare gli orari di apertura come lista di intervalli per giorno della settimana, anziché una singola finestra, per modellare correttamente chiusure infrasettimanali e pause pranzo (limite esplicitamente documentato in `data/SOURCES.md`).
3. Dataset completamente verificato: completare l'estrazione automatica da OpenStreetMap (`data/fetch_osm_pois.py`, predisposto ma non eseguibile nell'ambiente di sviluppo per restrizioni di rete) o la verifica manuale dei restanti POI, per consolidare le conclusioni dell'Esperimento B su un campione più ampio.
4. Rete Bayesiana per l'incertezza: integrare un modulo di ragionamento probabilistico (cfr. capitolo 9 del programma del corso) per gestire l'incertezza su variabili come le previsioni meteo o l'affollamento atteso, influenzando la selezione di POI indoor come riserva (idea delineata in `docs/PROJECT_PLAN.md` ma non implementata in questa versione).

In conclusione, TripSolver rappresenta un caso di studio applicato dell'integrazione fra tecniche simboliche e sub-simboliche viste nel corso, con un'attenzione esplicita, nella stesura di questo documento, a riportare risultati onesti — inclusi quelli che non confermano nettamente l'ipotesi di partenza — piuttosto che risultati ottimizzati a posteriori per apparire più convincenti.

9. Riferimenti Bibliografici

- [1] Russell, S. J., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- [2] Poole, D. L., & Mackworth, A. K. (2023). Artificial Intelligence: Foundations of Computational Agents (3rd ed.). Cambridge University Press.
- [3] Lamy, J. B. (2017). Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. Artificial Intelligence in Medicine, 80, 11-28.
- [4] Perron, L., & Furnon, V. Google OR-Tools (CP-SAT Solver). Disponibile su: <https://developers.google.com/optimization>
- [5] Scikit-learn Developers. (2023). User Guide: Supervised learning. Disponibile su: https://scikit-learn.org/stable/supervised_learning.html
- [6] Owlready2 Documentation. Disponibile su: <https://owlready2.readthedocs.io/>
- [7] OpenStreetMap Contributors. Dati geografici distribuiti con licenza ODbL. Disponibile su: <https://www.openstreetmap.org/copyright>
- [8] Comune di Bari / Musei Puglia / siti ufficiali dei singoli POI citati in data/SOURCES.md per la verifica degli orari di apertura (Basilica di San Nicola, Castello Svevo, Cattedrale di San Sabino, Pinacoteca Metropolitana, Museo Archeologico di Santa Scolastica, Teatro Margherita, Orto Botanico dell'Università di Bari, Acquario di Bari).